Working With Data DATA9910 Week 5

Lucas Rizzo

email: lucas.rizzo@tudublin.ie

OLLSCOIL TEICNEOLAÍOCHTA BHAILE ÁTHA CLIATH

TU DUBLIN

TECHNOLOGICAL UNIVERSITY DUBLIN

Indicative schedule

Week 1	Set up environment and R notebook
Week 2	Review Python and practice
Week 3	Review R, importing data
Week 4	Intro SQL and aggregate queries
Week 5	Databases and scripting languages **(To be submitted)**
Week 6	Interpreting Data (summary statistics, data visualisation)
Week 7	Review week
Week 8	Data manipulation in R Pandas and Numpy
Week 9	Data visualisation in Python *Data processing (regular expressions)*
Week 10	Audio data (R and tuneR) **(To be submitted)**
Week 11	Image data (Python and skimage) **(To be submitted)**
Week 12	Time series (Python and Pandas)
Week 13	Working on Assignment **Assignment deadline**

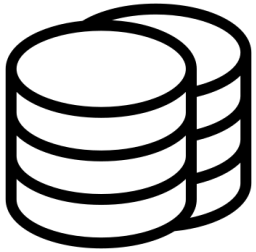
Overview

- Intro relationship between R/Python and SQL databases
- Establish connections to SQL databases
- Analyse structure and data format of SQL databases
- Retrieve data from SQL databases

Intro to Analytic Databases

- A (somewhat idealized) data lifecycle

1. Web Application generates data

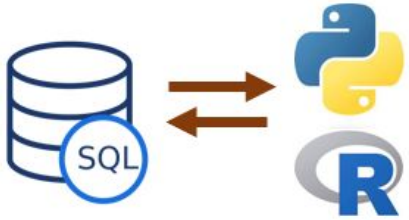


2. Data is stored in database

3. Data is extracted from database to flat file

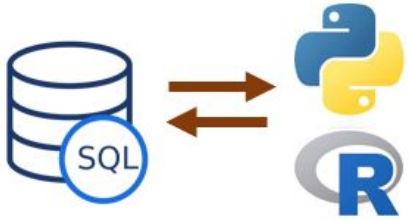
4. Data is processed in R / Python

Databases and Scripting Languages



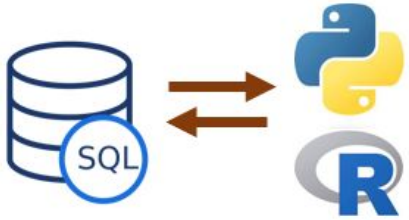
- The combination of databases and scripting languages offers a wide range of possibilities.
- With R and Python you can:
  - **Establish connections** to databases
  - **Retrieve** specific datasets
  - Perform **intricate data manipulations** without requiring extensive knowledge of database query languages:
    - Handle **missing values**
    - **Standardize formats**
    - **Restructure data** to prepare it for analysis
  - Perform **complex data analysis** and computations
  - Create **compelling visualizations**, allowing you to represent data trends, patterns, and relationships through graphs, charts, and interactive plots
  - Perform **iterative exploration**: retrieve data from databases, analyze it, gain insights, and then refine analysis based on findings

Databases and Scripting Languages
Drawbacks



- Some **drawbacks** can also arise from this combination:
  - Different databases use **varying data formats and structures**
  - **Performance bottlenecks** for very large datasets
  - **Security risks** for when transferring sensitive data
  - **Proficiency** in both databases and scripting languages

Databases and Scripting Languages
Alternatives

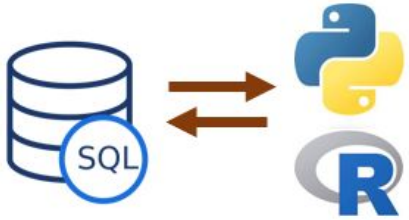


- Most database management systems offer a good range of analytical SQL functions, procedural languages, and ML capabilities that can be performed in the database.

DBMS	**Procedural languages**
PostgreSQL	PL/pgSQL (similar to Oracle's PL/SQL), PL/Python, PL/Tcl, and PL/Perl
Oracle Database	PL/SQL (specifically designed for database programming)
Microsoft SQL	Transact-SQL
MySQL	Stored program language
SQLite	C API

Examples of procedural languages for differents DBMs

Databases and Scripting Languages
Alternatives



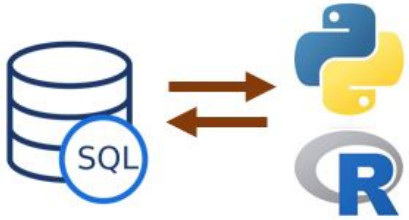
- Most database management systems offer a good range of analytical SQL functions, procedural languages, and ML capabilities that can be performed in the database.

Not in the scope of the course

DBMS	Procedural languages
PostgreSQL	PL/pgSQL (similar to Oracle's PL/SQL), PL/Python, PL/Tcl, and PL/Perl
Oracle Database	PL/SQL (specifically designed for database programming)
Microsoft SQL	Transact-SQL
MySQL	Stored program language
SQLite	C API

Examples of procedural languages for differents DBMs

Databases and Scripting Languages
Alternatives

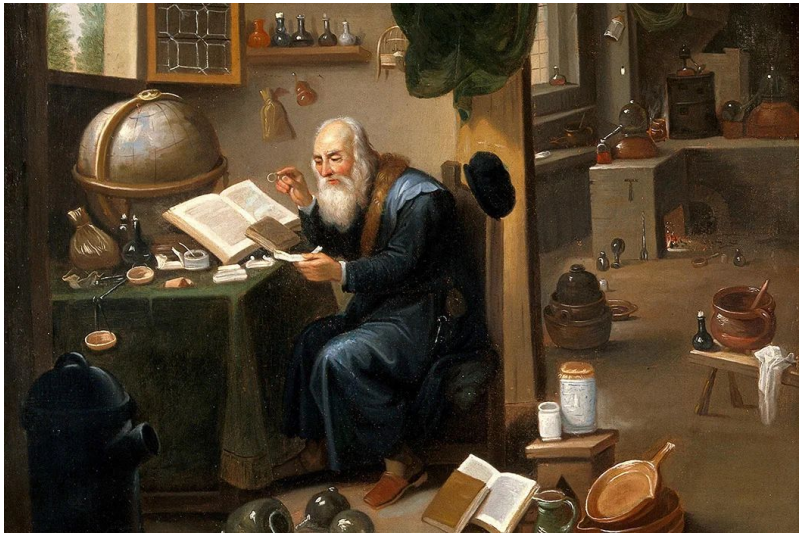



```
170
171 create table ac_dt_1_settings (
172     setting_name varchar2(30),
173     setting_value varchar2(30)
174 );
175 DECLARE
176     v_model_name varchar(30) := 'affinity_card_decision_tree_1';
177 BEGIN
178     delete from ac_dt_1_settings;
179
180     -- Use a Decision Tree (Naive Bayes is default)
181     insert into ac_dt_1_settings (setting_name, setting_value)
182     VALUES (dbms_data_mining.algo_name, dbms_data_mining.algo_decision_tree);
183
184     -- Specify that automatic data preparation should be carried out
185     insert into ac_dt_1_settings (setting_name, setting_value)
186     VALUES (dbms_data_mining.prep_auto, dbms_data_mining.prep_auto_on);
187
188     dbms_data_mining.drop_model(v_model_name);
189
190     DBMS_DATA_MINING.CREATE_MODEL(
191     model_name => v_model_name,
192     mining_function => dbms_data_mining.classification,
193     data_table_name => 'affinity_card_case_table',
194     case_id_column_name => 'CUST_ID',
195     target_column_name => 'AFFINITY_CARD',
196     settings_table_name => 'ac_dt_1_settings' -- uses default settings only
197 );
198 END;
```

Example of Oracle PL/SQL
Not in the scope of the course

SQLAlchemy

- For Python we are going to use the SQLAlchemy package.
  - Powerful and popular open-source SQL toolkit
- It allows users to:
  - Connect databases using Python language
  - Run SQL queries using object-based programming
  - Streamline the workflow.



SQLAlchemy

Establishing connections to PostgreSQL

```
import sqlalchemy as db
import psycopg2
from sqlalchemy import text

user = 'postgres'
password = 'admin@asd'
host = 'localhost'
port = 5432
database = 'students'

engine = db.create_engine('postgresql://{0}:{1}@{2}:{3}/{4}'.format(user,
                                                                    password,
                                                                    host,
                                                                    port,
                                                                    database))
conn = engine.connect()

result = conn.execute(text("SELECT * FROM student"))

# print all results
for row in result:
    print(row)
```

Establishing connections to PostgreSQL

```
result = conn.execute(text("SELECT * FROM student"))

# print all results
for row in result:
    print(row)
```

✓ 0.0s

```
('D020150120', 'James', 'Smith', datetime.date(1995, 1, 19), 'TU256')
('D020150121', 'John', 'Brown', datetime.date(1987, 9, 18), 'TU256')
('D020150122', 'Patricia', 'Wilson', datetime.date(1973, 10, 4), 'TU256')
('D020150123', 'Karen', 'Davies', datetime.date(2000, 12, 28), 'TU256')
```

Establishing connections to SQL databases

- Connection strings for other engines can be seen in the documentation.
- For example:
  - **SQLite**

```
# sqlite://<nohostname>/<path>
# where <path> is relative:
engine = create_engine("sqlite:///foo.db")
```

  - **Oracle**

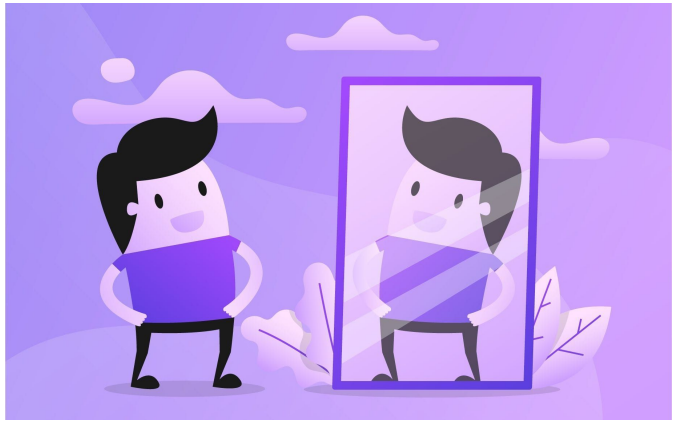
```
engine = create_engine("oracle://scott:tiger@127.0.0.1:1521/sidname")
engine = create_engine("oracle+cx_oracle://scott:tiger@tnsname")
```

Task 1

- Connect to the database created for the last lab and print the data in the student table

Loading tables

- SQLAlchemy can automatically load tables using something called **reflection**.
- Reflection is the process of **reading the database** and building its **metadata**.
- To perform reflection, you will first need to import and initialize a **MetaData object**.



MetaData object

- The MetaData object will be populated with information about the reflected tables automatically.

- It only needs to be **initialized before reflecting** by calling `MetaData()`.

```
import sqlalchemy as db
metadata_obj = db.MetaData() # initialize
metadata_obj.reflect(bind=engine) # reflect
```

MetaData object

- The MetaData supports a few methods of accessing these table objects.
- The `sorted_tables` method which returns a list of each table object in order of foreign key dependency.

```
import sqlalchemy as db
metadata_obj = db.MetaData()
metadata_obj.reflect(bind=engine)
for t in metadata_obj.sorted_tables:
    print(t)
```

```
course_codes
student
```

MetaData object

- You can also use the MetaData object to **modify the database**.

```
from sqlalchemy import Table, Column, Integer, String, MetaData

metadata_obj = db.MetaData()

user = Table(
    "user",
    metadata_obj,
    Column("user_id", Integer, primary_key=True),
    Column("user_name", String(16), nullable=False),
    Column("email_address", String(60), key="email"),
    Column("nickname", String(50), nullable=False),
)

user_prefs = Table(
    "user_prefs",
    metadata_obj,
    Column("pref_id", Integer, primary_key=True),
    Column("user_id", Integer, ForeignKey("user.user_id"), nullable=False),
    Column("pref_name", String(40), nullable=False),
    Column("pref_value", String(100)),
)

metadata_obj.create_all(engine)
```

MetaData object

- You can also use the MetaData object to **modify the database**.

```
from sqlalchemy import Table, Column, Integer, String, MetaData

metadata_obj = db.MetaData()

user = Table(
    "user",
    metadata_obj,
    Column("user_id", Integer, primary_key=True),
    Column("user_name", String(16), nullable=False),
    Column("email_address", String(60), key="email"),
    Column("nickname", String(50), nullable=False),
)

user_prefs = Table(
    "user_prefs",
    metadata_obj,
    Column("pref_id", Integer, primary_key=True),
    Column("user_id", Integer, ForeignKey("user.user_id"), nullable=False),
    Column("pref_name", String(40), nullable=False),
    Column("pref_value", String(100)),
)

metadata_obj.create_all(engine)
```

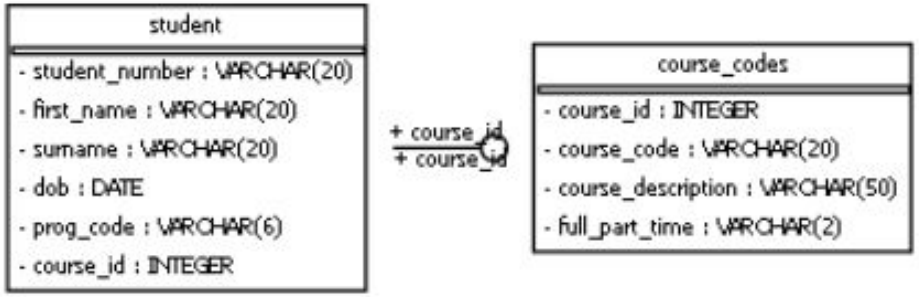
Not so relevant to us

Extracting the ERD

- Another Python module `sqlalchemy_schemadisplay` can be used to **turn SQLAlchemy DB Model into a graph.**

- It also requires `pydot` along with `graphviz` for this.

```
from sqlalchemy_schemadisplay import create_schema_graph

graph = create_schema_graph(engine=engine,
                            metadata=metadata_obj,
                            rankdir='LR') # Left to right
graph.write_png('erd.png')
```



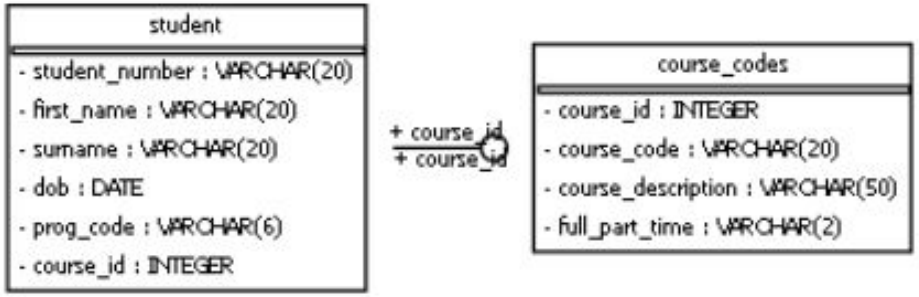
Extracting the ERD

- Another Python module `sqlalchemy_schemadisplay` can be used to **turn SQLAlchemy DB Model into a graph.**

- It also requires `pydot` along with `graphviz` for this.

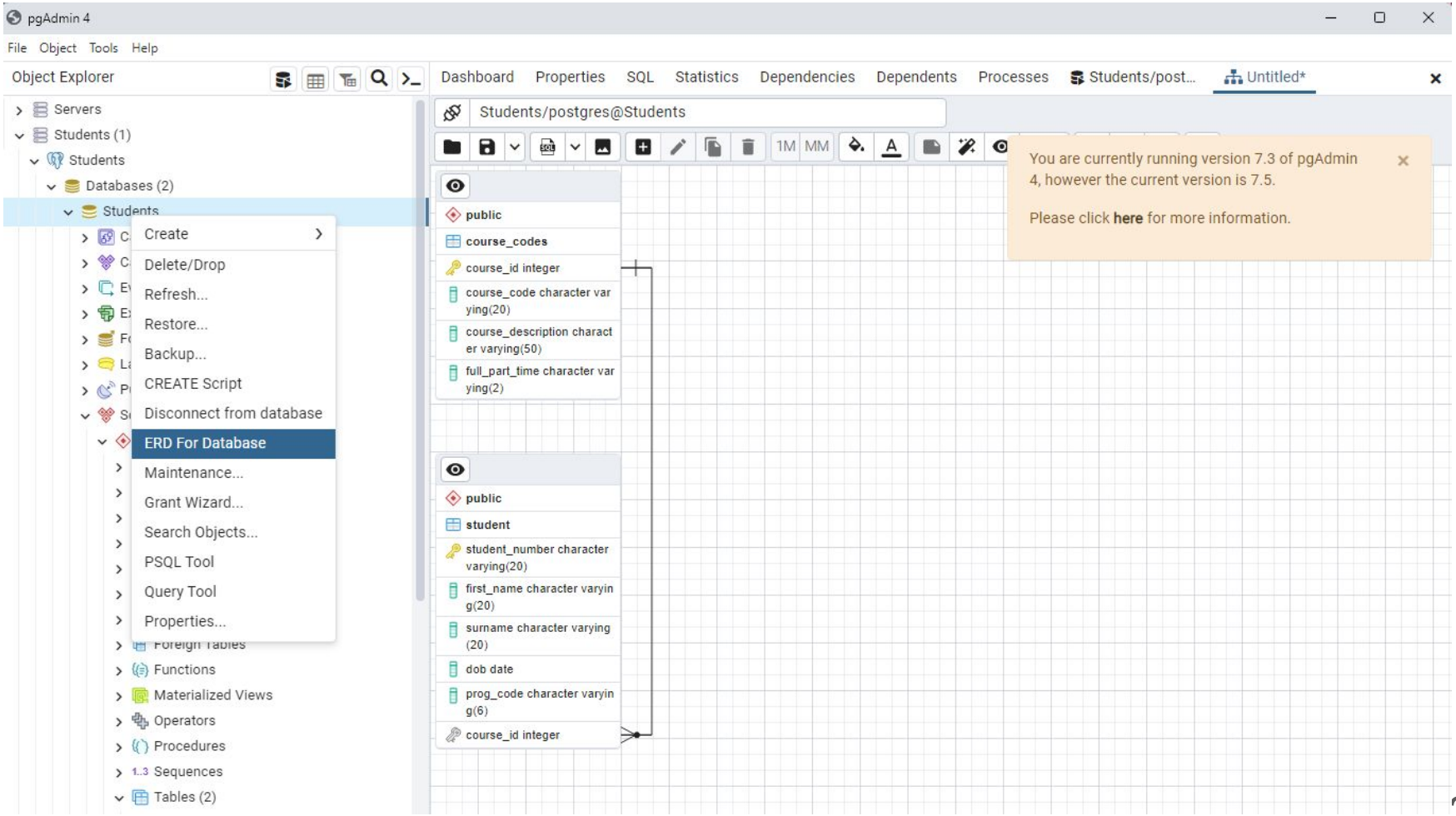
```
from sqlalchemy_schemadisplay import create_schema_graph

graph = create_schema_graph(engine=engine,
                            metadata=metadata_obj,
                            rankdir='LR') # Left to right
graph.write_png('erd.png')
```

It does not look amazing, but can be handy =)



Extracting the ERD

- The ERD can also be more easily extracted using PgAdmin



Selecting data (Raw query)

- Using the `.execute()` method of the connection, a raw SQL query can be performed.
- On the result object, the `.fetchall()` method can be called to get the actual data.

```
result = conn.execute(text("SELECT * FROM student")).fetchall()
for row in result:
    print(row)
```

✓ 0.0s

```
('D020150120', 'James', 'Smith', datetime.date(1995, 1, 19), 'TU256')
('D020150121', 'John', 'Brown', datetime.date(1987, 9, 18), 'TU256')
('D020150122', 'Patricia', 'Wilson', datetime.date(1973, 10, 4), 'TU256')
('D020150123', 'Karen', 'Davies', datetime.date(2000, 12, 28), 'TU256')
```

Selecting data (Raw query)

- `.fetchmany()` can also be used to limit the number of returned records.

```
result = conn.execute(text("SELECT * FROM student")).fetchmany(3)

# print top 3 results
for row in result:
    print(row)
```

✓ 0.0s

```
('D020150120', 'James', 'Smith', datetime.date(1995, 1, 19), 'TU256')
('D020150121', 'John', 'Brown', datetime.date(1987, 9, 18), 'TU256')
('D020150122', 'Patricia', 'Wilson', datetime.date(1973, 10, 4), 'TU256')
```

Task 2

- Select courses whose course description starts with `D`. OR
- Select `first_name` from students that starts with D

Task 2

- Select courses whose course description starts with `D`.
- Hint: You need to use `%%` instead of `%` for `LIKE` operations in python strings. A single `%` is used for string formatting.

```
Solution in the
next slide...
```

Task 2

- Select courses whose course description starts with `D`.
- Hint: You need to use `%%` instead of `%` for `LIKE` operations in python strings. A single `%` is used for string formatting.

```
result = conn.execute(text("SELECT * FROM course_codes WHERE course_description LIKE 'D%%'"))

# print top 3 results
for row in result:
    print(row)
```

✓ 0.0s

```
(1, 'TU001', 'Data Analytics', 'p')
(2, 'TU002', 'Data Science', 'p')
```

SQLAlchemy and Pandas

- We can feed a result set directly into a pandas DataFrame (more about pandas on future weeks)

```
result = conn.execute(text("SELECT * FROM course_codes")).fetchall()

data = pd.DataFrame(result)
data
```

✓ 0.0s

	course_id	course_code	course_description	full_part_time
0	1	TU001	Data Analytics	p
1	2	TU002	Data Science	p
2	3	TU003	Software Development	f
3	4	TU004	Computer Science	f

Selecting data (SQLAlchemy)

- SQLAlchemy also provides a nice "Pythonic" way of interacting with databases.
- You can interact with a **Table object** instead, and SQLAlchemy will translate your query to an appropriate SQL dialect.
- **Useful when working with multiple dialects** of SQL

Selecting data (SQLAlchemy)

- Get all table data with SQLAlchemy

```
student = db.Table('student', metadata_obj, autoload=True, autoload_with=engine)
stmt = db.select(student) # Select statement in SQLAlchemy
print(stmt)
print()
# Execute the statement on connection and fetch 10 records: results
results = conn.execute(stmt).fetchall()
print(results)
```

✓ 0.0s Python

```
SELECT student.student_number, student.first_name, student.surname, student.dob,
student.prog_code
FROM student

[('D020150120', 'James', 'Smith', datetime.date(1995, 1, 19), 'TU256'),
('D020150121', 'John', 'Brown', datetime.date(1987, 9, 18), 'TU256'),
('D020150122', 'Patricia', 'Wilson', datetime.date(1973, 10, 4), 'TU256'),
('D020150123', 'Karen', 'Davies', datetime.date(2000, 12, 28), 'TU256')]
```

Filtering data (SQLAlchemy)

- Filter table data by first name with SQLAlchemy

```
stmt = db.select(student) # Create a select query: stmt
stmt = stmt.where(student.columns.first_name == 'Lucas') # Add a where clause

results = conn.execute(stmt).fetchall()

# Loop over the results and print first name, last name, and dob
for result in results:
    print(result.first_name, result.surname, result.dob)
```

✓ 0.0s

```
Lucas Rizzo 1988-08-03
```

Filtering data (SQLAlchemy)

- Filter table data by first name in a set with SQLAlchemy

```
# Define a list of bad students
bad_students = ['James', 'John', 'Patricia']

stmt = db.select(student)
# Append a where clause to match all the students in_ the bad students list
stmt = stmt.where(student.columns.first_name.in_(bad_students))

# Loop over the ResultProxy and print the first and last names
for result in conn.execute(stmt):
    print(result.first_name, result.surname)
```

✓ 0.0s

```
James Smith
John Brown
Patricia Wilson
```

Filtering data (SQLAlchemy)

- Filter table data using boolean operators with SQLAlchemy
- You can use `and_()`, `or_()`, and `not_()`

```
# Define a list of bad students
bad_students = ['James', 'John', 'Patricia']

stmt = db.select(student)
# Append a where clause to match all the students in_ the bad students list
# that are in course_id 1
stmt = stmt.where(db.and_(student.columns.first_name.in_(bad_students),
                          student.columns.course_id == 1))

# Loop over the ResultProxy and print the first and last names
for result in conn.execute(stmt):
    print(result.first_name, result.surname)
```

✓ 0.0s

```
James Smith
John Brown
```

Joining data (SQLAlchemy)

- The `.join()` method can be used, with the right side of the join passed
- `isouter` – if True, renders a **LEFT OUTER JOIN**, instead of JOIN
- `full` – if True, renders a **FULL OUTER JOIN**, instead of LEFT OUTER JOIN

```
course_codes = db.Table('course_codes', metadata_obj, autoload=True, autoload_with=engine)

# Build a statement to join student and course_codes tables
stmt = db.select(student, course_codes)
stmt_join = stmt.select_from(
    student.join(course_codes, student.columns.course_id == course_codes.columns.course_id))

results = conn.execute(stmt_join)

# Loop over the results
for result in results:
    print(result.first_name, result.surname, "-", result.course_description)
```

✓ 0.0s

```
James Smith - Data Analytics
John Brown - Data Analytics
Patricia Wilson - Data Analytics
Karen Davies - Data Analytics
```

Example

- Using SQLAlchemy, we can implement the following raw SQL query

```
SELECT student.first_name, student.surname,
course_codes.course_description
FROM student JOIN course_codes
ON student.course_id = course_codes.course_id
AND course_codes.course_description LIKE 'D%';
```

Example

- Using SQLAlchemy, we can implement the following raw SQL query

```
SELECT student.first_name, student.surname, course_codes.course_description
FROM student JOIN course_codes
ON student.course_id = course_codes.course_id
AND course_codes.course_description LIKE 'D%';
```

```
course_codes = db.Table('course_codes', metadata_obj, autoload=True, autoload_with=engine)

# Build a statement to join student and course_codes tables
stmt = db.select(student, course_codes)
stmt_join = stmt.select_from(student.join(course_codes,
                                          db.and_(student.columns.course_id == course_codes.columns.course_id,
                                                  course_codes.columns.course_description.like("D%"))))

results = conn.execute(stmt_join)

# Loop over the results
for result in results:
    print(result.first_name, result.surname, "-", result.course_description)
```

✓ 0.0s

```
James Smith - Data Analytics
John Brown - Data Analytics
Patricia Wilson - Data Science
```

Selecting data (SQLAlchemy)

- In general **SQLAlchemy queries are quite powerful**.
- The documentation cover many more select examples and also insert, delete, update, etc.
- There **might be cases** where very complex or database-specific **queries are challenging to express** directly using SQLAlchemy.
- For our module, **raw PostgreSQL queries should be enough**, but you might want to work with data from different SQL dialects in the future.

What about R?

- In R the `DBI` package provides a consistent interface for connecting to and interacting with databases.
- It serves as a foundational layer for working with databases
- It's used by higher-level packages like `dplyr` and `dbplyr` to connect to databases and perform SQL operations.

What about R?

- To connect to a PostgreSQL database you will need the `RPostgres` package.

- DBI provides the connection method that allows us to pass the configuration parameters and established a connection.

```
# Connect to a PostgreSQL database
con <- dbConnect(RPostgres::Postgres(),
                 dbname = "Students",
                 host = "localhost",
                 port = 5432,
                 user = "postgres",
                 password = "admin")
```

What about R?

- With the connection object created, we can than pass queries and get results

```
result <- dbGetQuery(con, "SELECT * FROM course_codes WHERE course_description LIKE 'D%'")

result
```

A data.frame: 2 × 4

course_id	course_code	course_description	full_part_time
<int>	<chr>	<chr>	<chr>
1	TU001	Data Analytics	p
2	TU002	Data Science	p

What about R?

- The **dbplyr** package allows you to write R code that gets translated into SQL queries, similarly to what SQLAlchemy does in Python

- It is an extension of the **dplyr** package specifically designed to work with databases (more about dplyr later).

```r
# Create a remote data table using dbplyr
remote_table <- tbl(con, "student")

# Example queries using dbplyr

# Basic selection
query1 <- remote_table %>%
  select(first_name, surname)
query1
```

```
# Source:   SQL [4 x 2]
# Database: postgres  [postgres@localhost:5432/Students]
  first_name surname
  <chr>      <chr>
1 James      Smith
2 John       Brown
3 Patricia   Wilson
4 Lucas      Rizzo
```

What about R?

- Another difference compared to Python is that there is no need to reflect tables.

- We can for example use `dbListTables` passing the connection object to list all tables in the dataset.

```
# Build a vector of table names: tables
tables <- dbListTables(con)

# Display structure of tables
str(tables)
```

```
chr [1:2] "course_codes" "student"
```

What about R?

- Overall, both R and Python offer similar capability to extract data from SQL databases.
  - Both can connect to PostgreSQL and other SQL databases using dedicated libraries.
  - Both allow you to send queries, fetch results, and integrate data directly into your analysis workflow.

- There are syntax differences though, especially when working with **`dbplyr`** and **`sqlAlchemy`** packages to generate queries.

- Choice usually depends on language familiarity, ecosystem, and workflow preference.

Summary

- **Comparison** of **Python/R** and databases **procedural languages**
- **Establishing connections** to databases using SQLAlchemy
- Extracting **databases metadata** and showing ERD
- Running **raw SQL queries**
- Running **SQLAlchemy queries**

Lab

- You can see all the Python examples in the SQLAlchemy notebook.
- You can practice similar concepts using R and PostgreSQL in the R notebook.
- Complete the exercises in the end of the R notebook

Questions?